

Projet OPTICAL: Estimation du flot optique

UM4MA266

Fitahiry Rajaobelina

19 avril 2026

Table des matières

1	Introduction	3
1.1	Hypothèse de Constance de l'Intensité	3
1.1.1	Équation de Contrainte du Flot Optique (BCCE)	3
1.1.2	Le Problème d'Ouverture	3
1.2	La Méthode de Horn-Schunck	3
1.2.1	Fonctionnelle d'Énergie	3
1.2.2	Équations d'Euler-Lagrange	3
1.3	Approches Complémentaires	4
1.3.1	Méthode de Lucas-Kanade	4
1.3.2	Approche Multi-échelle	4
2	Implémentation de la méthode de Horn-Schunck	5
2.1	Discretisation des équations	5
2.2	Implémentation en Python	5
2.3	Étude de l'influence du paramètre α	6
2.3.1	D'un point de vue théorique	6
2.3.2	En pratique α	6
3	Comparaison avec la méthode de Lucas-Kanade	8
3.1	Résultats avec la méthode de Lucas-Kanade (Flot épars)	8
4	Approche Multi-échelle pour les Grands Déplacements	10
4.1	Limites de la linéarisation	10
4.2	Stratégie <i>Pyramide d'images</i>	10
4.3	Implémentation Python	10
4.4	Analyse des résultats de l'approche Multi-échelle	11
5	Tests sur séquences vidéo réelles	13
5.1	Cas d'un fluide en écoulement au robinet	13
5.2	Cas d'un fluide en rotation (Tourbillon de café)	14
5.3	Cas : Objet rigide en translation et rotation sur le sol (Balle de ping-pong)	14
5.4	Bilan des expérimentations sur vidéos réelles	15
6	Extension : Préservation des discontinuités par la Variation Totale (Norme L^1)	16
6.1	Limites de la régularisation de Tikhonov (L^2)	16
6.2	Défi technique et implémentation Python	16
6.3	Évaluation empirique du modèle TV- L^1 et comparaison avec le modèle L^2	18

1 Introduction

Le flot optique est défini comme le champ de vecteurs décrivant le mouvement apparent des pixels entre deux images successives d'une séquence vidéo. Mathématiquement, pour un pixel situé à (x, y) à l'instant t , nous cherchons le vecteur de déplacement (u, v) tel que l'intensité lumineuse soit préservée à l'instant $t + 1$.

1.1 Hypothèse de Constance de l'Intensité

1.1.1 Équation de Contrainte du Flot Optique (BCCE)

L'hypothèse fondamentale est qu'un point de la scène conserve son intensité lumineuse au cours du temps :

$$I(x, y, t) = I(x + u, y + v, t + 1) \quad (1)$$

En effectuant un développement de Taylor au premier ordre de la partie droite de l'équation (1) autour de (x, y, t) , nous obtenons :

$$I_x u + I_y v + I_t = 0 \quad (2)$$

où $I_x = \frac{\partial I}{\partial x}$, $I_y = \frac{\partial I}{\partial y}$ sont les gradients spatiaux et $I_t = \frac{\partial I}{\partial t}$ est la dérivée temporelle.

1.1.2 Le Problème d'Ouverture

L'équation (2) est une équation unique pour deux inconnues (u, v) en chaque pixel. Par conséquent, le système est sous-déterminé, ce qui signifie que nous ne pouvons déterminer que la composante du mouvement normale aux contours de l'image. C'est ce qu'on appelle le *problème d'ouverture* (aperture problem). Des contraintes de régularisation sont donc nécessaires pour lever cette ambiguïté.

1.2 La Méthode de Horn-Schunck

1.2.1 Fonctionnelle d'Énergie

Pour résoudre le problème d'ouverture, Horn et Schunck proposent d'ajouter une contrainte de régularisation spatiale favorisant un champ de vecteurs "lisse". On cherche à minimiser la fonctionnelle d'énergie suivante :

$$E(u, v) = \iint_{\Omega} [(I_x u + I_y v + I_t)^2 + \alpha(|\nabla u|^2 + |\nabla v|^2)] dx dy \quad (3)$$

où :

- $(I_x u + I_y v + I_t)^2$ est le terme de fidélité aux données (respect de la BCCE).
- $|\nabla u|^2 + |\nabla v|^2$ est le terme de régularisation (lissage).
- $\alpha > 0$ est le paramètre de pondération entre ces deux termes.

1.2.2 Équations d'Euler-Lagrange

La minimisation de la fonctionnelle $E(u, v)$ par le calcul des variations conduit au système d'équations aux dérivées partielles (EDP) suivant :

$$I_x(I_x u + I_y v + I_t) - \alpha \Delta u = 0 \quad (4)$$

$$I_y(I_x u + I_y v + I_t) - \alpha \Delta v = 0 \quad (5)$$

où $\Delta = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ représente l'opérateur Laplacien.

1.3 Approches Complémentaires

1.3.1 Méthode de Lucas-Kanade

Contrairement à Horn-Schunck qui est une méthode globale, Lucas-Kanade suppose que le flot est constant dans un petit voisinage local $\mathcal{W}$ autour de chaque pixel. Le problème revient alors à résoudre un système de moindres carrés :

$$\min_{u,v} \sum_{(x_i,y_i) \in \mathcal{W}} w_i (I_x(x_i, y_i)u + I_y(x_i, y_i)v + I_t(x_i, y_i))^2 \quad (6)$$

1.3.2 Approche Multi-échelle

Pour estimer des déplacements importants (supérieurs à un pixel), l'hypothèse de linéarité du développement de Taylor n'est plus valide. On utilise alors une pyramide d'images (pyramid scale) pour estimer le mouvement de manière grossière à basse résolution, puis on l'affine progressivement aux résolutions supérieures.

2 Implémentation de la méthode de Horn-Schunck

2.1 Discrétisation des équations

Pour résoudre numériquement le système d'Euler-Lagrange, nous devons discrétiser l'opérateur Laplacien Δ . Une approximation classique par différences finies sur une grille régulière de pixels donne :

$$\Delta u \approx \bar{u} - u \quad (7)$$

$$\Delta v \approx \bar{v} - v \quad (8)$$

où $\bar{u}$ et $\bar{v}$ représentent les moyennes locales des vitesses au voisinage du pixel considéré, souvent calculées à l'aide d'un noyau de convolution.

En substituant cette approximation dans les équations d'Euler-Lagrange obtenues précédemment (4) (5), nous avons :

$$I_x(I_x u + I_y v + I_t) - \alpha(\bar{u} - u) = 0 \quad (9)$$

$$I_y(I_x u + I_y v + I_t) - \alpha(\bar{v} - v) = 0 \quad (10)$$

En isolant u et v , on aboutit au schéma itératif de Jacobi. Ce schéma permet de mettre à jour le flot optique à chaque itération k :

$$u^{k+1} = \bar{u}^k - I_x \frac{I_x \bar{u}^k + I_y \bar{v}^k + I_t}{\alpha + I_x^2 + I_y^2} \quad (11)$$

$$v^{k+1} = \bar{v}^k - I_y \frac{I_x \bar{u}^k + I_y \bar{v}^k + I_t}{\alpha + I_x^2 + I_y^2} \quad (12)$$

2.2 Implémentation en Python

Le code suivant implémente ce schéma itératif. Nous utilisons la fonction `convolve` de la bibliothèque `scipy.ndimage` pour appliquer efficacement les filtres de calcul de gradients spatio-temporels et de moyennes locales.

```
1 import numpy as np
2 import cv2
3 from scipy.ndimage import convolve
4
5 def compute_derivatives(im1, im2):
6     """
7     Calcule les gradients spatio-temporels (Ix, Iy, It).
8     """
9     # Noyaux de Horn-Schunck
10    kernel_X = np.array([[ -1,  1], [ -1,  1]]) * 0.25
11    kernel_Y = np.array([[ -1, -1], [ 1,  1]]) * 0.25
12    kernel_T = np.ones((2, 2)) * 0.25
13
14    # Convolutions
15    Ix = convolve(im1, kernel_X) + convolve(im2, kernel_X)
16    Iy = convolve(im1, kernel_Y) + convolve(im2, kernel_Y)
17    It = convolve(im2, kernel_T) - convolve(im1, kernel_T)
18
19    return Ix, Iy, It
20
21 def horn_schunck(im1, im2, alpha=1.0, num_iter=100):
22     """
23     Estime le flot optique par la methode de Horn-Schunck.
24     """
25    im1 = im1.astype(np.float32) / 255.0
```

```

26     im2 = im2.astype(np.float32) / 255.0
27
28     Ix, Iy, It = compute_derivatives(im1, im2)
29
30     u = np.zeros_like(im1)
31     v = np.zeros_like(im1)
32
33     # Noyau pour la moyenne locale
34     kernel_avg = np.array([[1/12, 1/6, 1/12],
35                             [1/6, 0, 1/6],
36                             [1/12, 1/6, 1/12]], dtype=np.float32)
37
38     # Boucle itérative de Jacobi
39     for _ in range(num_iter):
40         u_avg = convolve(u, kernel_avg)
41         v_avg = convolve(v, kernel_avg)
42
43         numerator = (Ix * u_avg + Iy * v_avg + It)
44         denominator = (alpha + Ix**2 + Iy**2)
45
46         update_term = numerator / denominator
47
48         u = u_avg - Ix * update_term
49         v = v_avg - Iy * update_term
50
51     return u, v

```

Listing 1 – Implémentation de la méthode de Horn-Schunck avec le schéma de Jacobi

2.3 Étude de l'influence du paramètre α

2.3.1 D'un point de vue théorique

La méthode de Horn-Schunck repose sur la minimisation d'une fonctionnelle d'énergie globale composée de deux termes :

$$E(u, v) = E_{\text{data}}(u, v) + \alpha E_{\text{smooth}}(u, v) \quad (13)$$

Le paramètre $\alpha > 0$ joue le rôle de multiplicateur de Lagrange et contrôle le compromis entre fidélité aux données et régularité. L'étude empirique de ce paramètre met en évidence deux comportements asymptotiques :

- **Pour un α faible (ex : $\alpha = 0.1$)** : Le terme E_{data} domine. L'algorithme cherche à respecter scrupuleusement l'hypothèse de constance de l'intensité. Le champ de flot obtenu est très sensible au bruit local de l'image et aux variations d'illumination. Le résultat est souvent bruité mais préserve bien les discontinuités nettes du mouvement.
- **Pour un α élevé (ex : $\alpha = 50$ ou 100)** : Le terme de lissage E_{smooth} domine. L'algorithme pénalise fortement les variations spatiales du flot optique (∇u et ∇v). Le champ de vecteurs résultant est très régulier et lisse. Cependant, un α trop élevé entraîne un phénomène de sur-lissage (over-smoothing), effaçant les frontières réelles entre les objets en mouvement et le fond (perte des discontinuités).

2.3.2 En pratique α

La figure 3 présente l'estimation du flot optique entre deux images successives (voir figures 1 et 2) pour différentes valeurs du paramètre de régularisation α . Les deux images ont été capturées comme suit : on déplace latéralement l'appareil photo vers la gauche. L'analyse de ces résultats permet de mettre en évidence le compromis entre l'attache aux données et la régularité du champ de vecteurs.



FIGURE 1 – Image de référence (Frame 1)



FIGURE 2 – Image suivante (Frame 2)

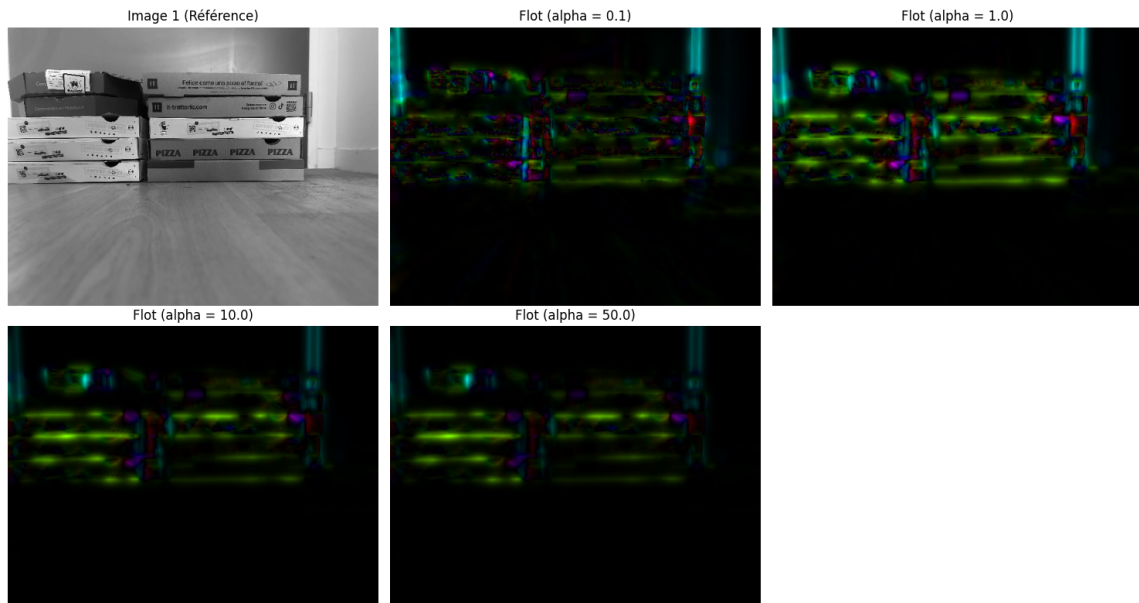


FIGURE 3 – Comparaison du flot optique pour différentes valeurs de α (Visualisation HSV)

Analyse détaillée

- **Faible régularisation** ($\alpha = 0.1$) : Le champ de flot est extrêmement bruité et haché. L'algorithme privilégie presque exclusivement le terme de *data fidelity*, ce qui rend le système instable face au bruit d'acquisition du capteur et aux légères variations d'illumination. Bien que les contours des objets soient visibles, le mouvement n'est pas cohérent spatialement.
- **Compromis optimal** ($\alpha = 1.0$ à 10.0) : On observe une disparition progressive du bruit. Le mouvement global de la pile de cartons devient homogène. Le terme de lissage permet de propager l'information de mouvement des zones à fort gradient (les bords et inscriptions sur les boîtes) vers les zones plus uniformes. C'est ici que l'on obtient la meilleure représentation physique du déplacement.
- **Forte régularisation** ($\alpha = 50.0$) : On assiste à un phénomène de **sur-lissage** (*over-smoothing*). Le champ de vecteurs devient très flou et s'étend bien au-delà des limites physiques des objets en mouvement. La contrainte de régularité est si forte qu'elle écrase les discontinuités naturelles du mouvement entre les cartons et le sol statique.

Conclusion : L'utilisation de la norme L^2 dans la fonctionnelle de Horn-Schunck tend à lisser

les discontinuités. Le choix de α est donc crucial et doit être ajusté pour équilibrer la robustesse au bruit et la précision des contours de mouvement.

3 Comparaison avec la méthode de Lucas-Kanade

Contrairement à la méthode de Horn-Schunck qui estime un flot optique *dense* (un vecteur en chaque pixel) à l'aide d'une régularisation globale, la méthode de Lucas-Kanade est une approche *locale*. Elle repose sur l'hypothèse que le déplacement (u, v) est constant dans un petit voisinage spatial $\mathcal{W}$ autour du point d'intérêt.

3.1 Résultats avec la méthode de Lucas-Kanade (Flot épars)

La figure 4 illustre l'application de l'algorithme de Lucas-Kanade sur la même paire d'images. Contrairement à la méthode de Horn-Schunck, cette approche locale génère un flot optique *épars* (sparse).



FIGURE 4 – Estimation du flot optique épars par la méthode de Lucas-Kanade. Les points rouges indiquent les points d'intérêt suivis, et les lignes vertes représentent leurs vecteurs de déplacement.

Interprétation des résultats

L'analyse visuelle de cette figure permet de tirer plusieurs conclusions sur le comportement de l'algorithme de Lucas-Kanade en conditions réelles :

- **Sélection des points d'intérêt** : On observe que les points de suivi (en rouge) ne sont pas répartis uniformément. Ils se concentrent exclusivement sur les zones présentant de forts gradients spatiaux dans plusieurs directions, comme les coins des boîtes, les logos (ex : "PIZZA") et les textes imprimés.

- **Robustesse au problème d'ouverture** : Les zones de texture uniforme (comme le sol en parquet, le mur en arrière-plan, ou les faces lisses des cartons) sont totalement ignorées par l'algorithme. Lucas-Kanade nécessite une matrice de structure locale (gradient) inversible ; en ignorant ces zones plates, la méthode évite l'instabilité numérique et le fameux problème d'ouverture auquel Horn-Schunck devait pallier par une régularisation globale.
- **Cohérence du mouvement** : La majorité des vecteurs de déplacement (lignes vertes) pointent de manière cohérente vers la droite, capturant correctement la translation physique des objets entre les deux trames. Étant calculé localement, le mouvement de chaque point n'influence pas ses voisins, ce qui préserve parfaitement les discontinuités (le fond reste statique).
- **Limites et vecteurs aberrants** : On peut remarquer quelques erreurs de correspondance, notamment au centre de l'image où certains vecteurs verts semblent croiser d'autres boîtes ou pointer dans des directions incohérentes. Cela illustre la principale faiblesse de Lucas-Kanade : si le déplacement entre deux images est supérieur à la taille de la fenêtre locale $\mathcal{W}$, ou si des motifs se répètent, l'algorithme peut se "perdre" et associer un point à une mauvaise cible.

Conclusion comparative : Lucas-Kanade est une méthode extrêmement rapide et précise sur les points distinctifs, idéale pour le suivi d'objets ou la stabilisation vidéo. Cependant, pour obtenir une carte de profondeur ou comprendre le mouvement de l'ensemble des pixels (notamment sur des surfaces peu texturées), une méthode dense comme Horn-Schunck (ou l'approche de Farneback) reste indispensable.

4 Approche Multi-échelle pour les Grands Déplacements

4.1 Limites de la linéarisation

L'équation de contrainte du flot optique (BCCE) repose sur un développement de Taylor au premier ordre. Mathématiquement, cela implique que le déplacement (u, v) entre les instants t et $t + 1$ doit être suffisamment petit (idéalement inférieur à un pixel) pour que l'approximation linéaire soit valide. Face à des mouvements rapides ou de grands déplacements, cette hypothèse s'effondre, générant des erreurs importantes et un phénomène de repliement (aliasing spatial).

4.2 Stratégie *Pyramide d'images*

Pour gérer les grands déplacements, l'approche standard consiste à utiliser une pyramide d'images : on estime d'abord le flot à basse résolution, puis on l'affine progressivement aux résolutions supérieures. L'algorithme se décompose ainsi :

1. **Construction de la pyramide** : Les images originales sont successivement floutées (filtre gaussien) et sous-échantillonnées par un facteur 2 pour créer L niveaux de résolution. À un niveau élevé (basse résolution), un grand déplacement physique de plusieurs pixels se réduit à un mouvement sous-pixelique.
2. **Estimation initiale** : Le flot optique est d'abord calculé au niveau le plus grossier (sommet de la pyramide) en utilisant la méthode de Horn-Schunck standard.
3. **Projection et recalage** : Le flot estimé est projeté (sur-échantillonné) vers le niveau de résolution supérieur. Plutôt que de recalculer le flot depuis zéro, on utilise ce flot grossier pour recalculer la seconde image vers la première.
4. **Calcul du flot résiduel** : On applique à nouveau Horn-Schunck entre la première image et l'image déformée pour calculer un incrément de flot $(\Delta u, \Delta v)$. Le flot total est mis à jour en additionnant le flot projeté et le résidu.

Ce processus itératif est répété jusqu'à atteindre la résolution originale (base de la pyramide).

4.3 Implémentation Python

Le code suivant implémente cette logique multi-échelle en s'appuyant sur notre fonction `horn_schunck` codée précédemment. La fonction `warp_image` est responsable du recalage de la seconde image à chaque niveau de la pyramide.

```
1 import cv2
2 import numpy as np
3
4 def warp_image(img, u, v):
5     """
6     Deforme (warp) l'image 'img' en utilisant le champ de flot (u, v).
7     """
8     h, w = img.shape
9     x, y = np.meshgrid(np.arange(w), np.arange(h))
10
11     map_x = (x + u).astype(np.float32)
12     map_y = (y + v).astype(np.float32)
13
14     warped_img = cv2.remap(img, map_x, map_y, interpolation=cv2.INTER_LINEAR,
15                             borderMode=cv2.BORDER_REFLECT)
16     return warped_img
17
18 def multiscale_horn_schunck(im1, im2, num_levels=3, alpha=1.0, num_iter=50):
19     """
20     Implemente la methode multi-echelle coarse-to-fine.
```

```

21 # 1. Construction des pyramides
22 pyramid1 = [im1.astype(np.float32) / 255.0]
23 pyramid2 = [im2.astype(np.float32) / 255.0]
24
25 for i in range(1, num_levels):
26     pyramid1.append(cv2.pyrDown(pyramid1[-1]))
27     pyramid2.append(cv2.pyrDown(pyramid2[-1]))
28
29 # Initialisation du flot a zero au sommet de la pyramide
30 u = np.zeros(pyramid1[-1].shape, dtype=np.float32)
31 v = np.zeros(pyramid2[-1].shape, dtype=np.float32)
32
33 # 2. Descente de la pyramide
34 for level in range(num_levels - 1, -1, -1):
35     img1_lvl = pyramid1[level]
36     img2_lvl = pyramid2[level]
37
38     if level != num_levels - 1:
39         # Sur-echantillonnage du flot et multiplication par 2
40         u = cv2.pyrUp(u) * 2.0
41         v = cv2.pyrUp(v) * 2.0
42
43         # Ajustement strict des dimensions (largeur, hauteur)
44         u = cv2.resize(u, (img1_lvl.shape[1], img1_lvl.shape[0]))
45         v = cv2.resize(v, (img1_lvl.shape[1], img1_lvl.shape[0]))
46
47     # Recalage de l'image 2
48     img2_warped = warp_image(img2_lvl, u, v)
49
50     # Calcul du residu
51     du, dv = horn_schunck(img1_lvl * 255, img2_warped * 255, alpha=alpha,
52                           num_iter=num_iter)
53
54     # Mise a jour
55     u += du
56     v += dv
57
58 return u, v

```

Listing 2 – Implémentation de l’approche multi-échelle pour Horn-Schunck

4.4 Analyse des résultats de l’approche Multi-échelle

La figure 5 présente les résultats de l’estimation du flot optique en utilisant la pyramide gaussienne (approche *coarse-to-fine*) sur notre séquence de test présentant un déplacement significatif.



FIGURE 5 – Résultat de l’algorithme de Horn-Schunck multi-échelle. De gauche à droite : Image de référence, Image suivante, et Flot optique estimé (visualisation HSV).

Interprétation visuelle et technique

L'analyse de cette visualisation met en exergue plusieurs améliorations majeures par rapport aux méthodes précédentes :

- **Gestion des grands déplacements** : Contrairement à l'algorithme de Horn-Schunck standard qui produisait des vecteurs aberrants ou du bruit face à un mouvement rapide, l'approche multi-échelle parvient à estimer un champ de vecteurs cohérent. En résolvant d'abord le mouvement à basse résolution, les grands déplacements en pixels sont réduits à des déplacements sous-pixelles, respectant ainsi l'hypothèse de linéarisation de l'équation de Taylor.
- **Densité et Cohérence spatiale** : On retrouve la nature *dense* de Horn-Schunck (tout l'objet en mouvement est coloré, et non pas seulement les coins comme avec Lucas-Kanade). Les cartons de pizza affichent des teintes homogènes (dominante de vert/violet selon la direction), ce qui indique que l'algorithme a bien compris que ces pixels appartiennent à un même solide en translation.
- **Préservation des contours (Bords nets)** : C'est le grand succès de cette méthode. Sur la version mono-échelle, il fallait augmenter drastiquement α pour lisser le bruit, ce qui floutait complètement les bords des cartons. Ici, grâce à la propagation du flot de la pyramide, nous pouvons garder un α modéré lors du calcul du flot résiduel. Résultat : la séparation entre les boîtes en mouvement et le sol statique (en noir) est beaucoup plus nette et respecte la géométrie de la scène.
- **Limites persistantes** : On observe tout de même quelques artefacts (notamment sur le bord droit de l'image ou dans les zones d'ombre uniformes). Ces erreurs résiduelles sont typiques des zones souffrant du problème d'ouverture ou d'occlusions, que même l'approche multi-échelle par différences finies peine à résoudre parfaitement sans un terme de régularisation plus complexe (comme la Variation Totale L^1 , qui fera l'objet de l'extension).

Conclusion : L'implémentation multi-échelle valide notre hypothèse théorique. Elle offre le meilleur compromis observé jusqu'ici : un flot dense, robuste aux déplacements importants, et qui préserve raisonnablement les discontinuités de mouvement.

5 Tests sur séquences vidéo réelles

5.1 Cas d'un fluide en écoulement au robinet

Pour valider nos algorithmes en conditions réelles, nous avons traité le flux continu d'une vidéo filmée au smartphone illustrant un écoulement d'eau depuis un robinet. Ce cas d'usage est particulièrement intéressant car la dynamique des fluides présente des défis majeurs pour la vision par ordinateur.

Les figures 6 et 7 comparent les résultats obtenus par l'algorithme de Farnebäck (intégré dans OpenCV) et notre implémentation de Horn-Schunck multi-échelle.

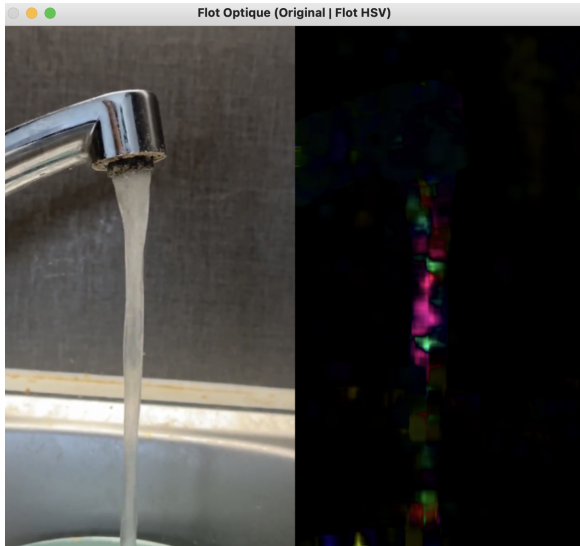


FIGURE 6 – Flot optique estimé par la méthode de Farnebäck (OpenCV).

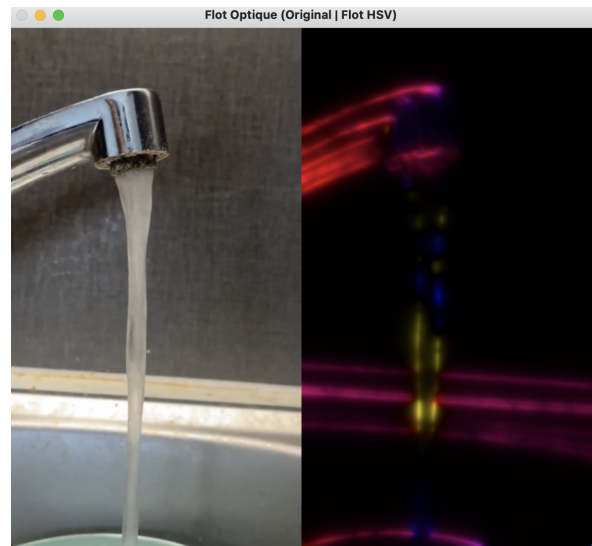


FIGURE 7 – Flot optique estimé par Horn-Schunck multi-échelle.

Interprétation des résultats

L'analyse de ces résultats met en évidence la violation des hypothèses fondamentales du flot optique dans le cas des fluides :

- **Violation de la constance d'intensité** : L'eau étant transparente et réfléchissante, l'hypothèse de base $I(x, y, t) = I(x + u, y + v, t + 1)$ n'est pas respectée. Les reflets dynamiques de l'eau sur la surface chromée du robinet et sur l'évier agissent comme des variations d'intensité trompeuses.
- **Robustesse de la méthode de Farnebäck** : L'algorithme de Farnebäck (Figure 6) parvient à isoler assez fidèlement le jet d'eau. En approximant les voisinages des pixels par des expansions polynomiales quadratiques plutôt que par de simples gradients linéaires, cette méthode se montre beaucoup plus résiliente face aux textures non rigides et fluides. Le fond statique est correctement ignoré.
- **Artefacts de la méthode Horn-Schunck multi-échelle** : Sur la figure 7, on observe de sévères aberrations (grandes traînées colorées sur le bras du robinet et le bord de l'évier). Le mouvement de l'eau crée des reflets spéculaires sur le métal. L'algorithme interprète ces changements de lumière locaux comme un déplacement massif. Le terme de régularisation globale (α) force ensuite ce faux mouvement à se propager. Enfin, l'approche *coarse-to-fine* exacerbe le problème : une erreur d'interprétation des reflets au niveau le plus grossier de la pyramide est propagée et amplifiée lors du sur-échantillonnage, créant ces larges zones de "flot fantôme".

5.2 Cas d'un fluide en rotation (Tourbillon de café)

Pour compléter notre analyse sur les fluides, nous avons testé nos algorithmes sur un mouvement complexe : le tourbillonnement d'un café dans une tasse blanche, généré par une cuillère. Le fond (un évier métallique) est supposé statique.

Les figure 8 et 9 présentent la comparaison entre la méthode de Farnebäck et notre implémentation de Horn-Schunck multi-échelle sur cette séquence.

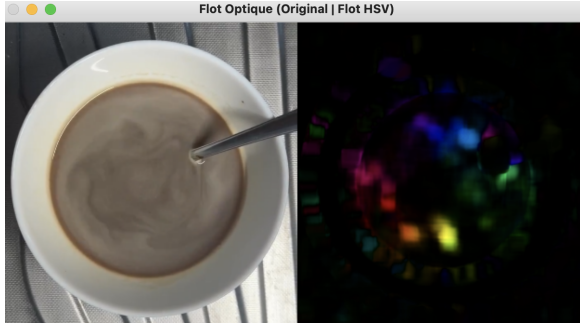


FIGURE 8 – Flot optique estimé par Farnebäck.

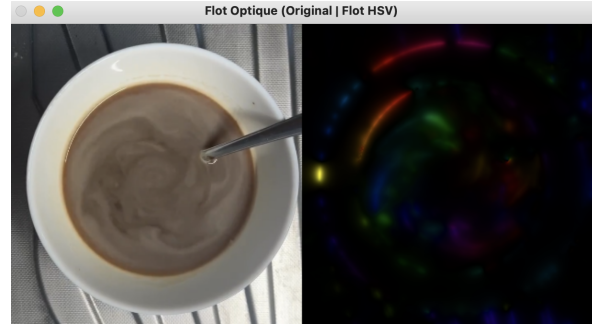


FIGURE 9 – Flot optique estimé par Horn-Schunck multi-échelle.

Interprétation des résultats

Cette expérience met en lumière la façon dont chaque algorithme gère les champs de vecteurs continus et les discontinuités spatiales (le bord de la tasse) :

- **Détection de la rotation (Code couleur HSV)** : Les deux algorithmes réussissent à identifier la nature rotative du mouvement. On observe un anneau contenant toutes les couleurs de l'arc-en-ciel (rouge, jaune, vert, bleu, violet). Dans l'espace HSV, la teinte représente l'angle du vecteur. Cette variation continue de la couleur confirme que le champ de flot pointe bien de manière tangentielle le long du tourbillon.
- **Le bruit local (Farnebäck)** : Sur la figure 8, le mouvement interne du liquide est détecté, mais il est assez morcelé. Surtout, on remarque que l'algorithme de Farnebäck est perturbé par le fond : il détecte du "faux" mouvement sur les stries métalliques de l'évier en arrière-plan (taches colorées), probablement à cause de légers changements de luminosité ou de micro-vibrations de la caméra.
- **L'effet de "Halo" et le lissage L^2 (Horn-Schunck Multi-échelle)** : Sur la figure 9, le mouvement interne du café est beaucoup plus lisse et continu grâce au terme de régularisation. Le fond est également mieux ignoré (plus noir). Cependant, on observe un défaut majeur typique de Horn-Schunck : un fort anneau lumineux apparaît exactement sur le bord en porcelaine de la tasse (qui est pourtant parfaitement immobile).
- **Explication mathématique de l'artefact** : Ce "halo" sur le bord de la tasse illustre la limite de la norme L^2 utilisée dans Horn-Schunck. La frontière entre le liquide sombre en mouvement et la tasse blanche statique crée un gradient spatial extrêmement fort. La régularisation quadratique $\alpha(|\nabla u|^2 + |\nabla v|^2)$ pénalise excessivement les sauts brusques du flot optique. Pour minimiser cette énergie, l'algorithme préfère étaler mathématiquement le mouvement du liquide vers la tasse immobile, créant cette illusion de mouvement sur la porcelaine.

5.3 Cas : Objet rigide en translation et rotation sur le sol (Balle de ping-pong)

Pour ce dernier test sur vidéo réelle, nous avons capturé le mouvement rapide d'une balle de ping-pong roulant sur un parquet en bois. Cette séquence permet d'évaluer la capacité des algo-

rithmes à dissocier un objet mobile d'un fond fortement texturé, et à capturer une cinématique complexe (translation couplée à une rotation).

Les figures 10 et 11 comparent l'approche épars (Lucas-Kanade) et l'approche dense (Horn-Schunck multi-échelle).

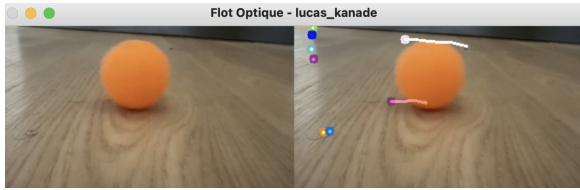


FIGURE 10 – Flot optique épars (Lucas-Kanade).

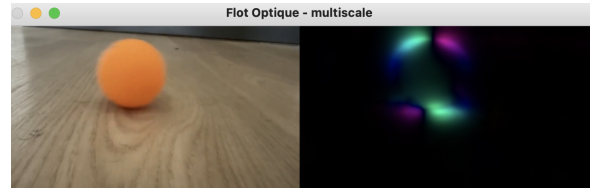


FIGURE 11 – Flot optique dense (Horn-Schunck multi-échelle).

Interprétation des résultats

L'analyse de ces deux comportements met en évidence les avantages et les limites de chaque algorithme :

- **Le suivi de caractéristiques par Lucas-Kanade** : Sur la figure 10, l'algorithme a détecté des points d'intérêt non seulement sur la balle, mais aussi sur le parquet et le mur (zones à fort gradient). Les points situés sur le fond statique restent immobiles (simples points colorés sans traînée), tandis que les points situés sur la balle affichent des trajectoires nettes suivant la translation. Lucas-Kanade est donc très efficace pour extraire des trajectoires précises, mais il ne segmente pas l'objet : il se contente de suivre des "coins" où qu'ils soient.
- **Segmentation parfaite par Horn-Schunck multi-échelle** : La figure 11 démontre la puissance de l'approche dense. Le fond (le parquet) est parfaitement noir, ce qui indique que l'algorithme a correctement estimé un mouvement nul sur ces pixels, malgré la forte texture du bois. La balle se détache parfaitement du fond. L'approche multi-échelle permet de corriger efficacement le grand déplacement.
- **Capture de la cinématique de roulement** : Une observation intéressante sur la figure 11 concerne les couleurs de la balle. Contrairement aux boîtes de pizza (qui étaient d'une couleur unie car en pure translation), la balle présente un dégradé complexe de teintes (cyan, vert, magenta). Cela s'explique par la physique du mouvement : la balle *roule*. La vitesse apparente des pixels au sommet de la balle n'est pas la même que celle des pixels à la base ou sur les côtés (projection 2D d'une rotation 3D). Le flot dense parvient à capturer cette distribution complexe des vitesses sur la surface de la sphère.

5.4 Bilan des expérimentations sur vidéos réelles

Plusieurs conclusions fondamentales se dégagent de ces expérimentations :

L'efficacité du suivi local (Lucas-Kanade) : Très rapide et numériquement stable grâce à la sélection préalable des zones fortement texturées (coins), cette méthode excelle pour extraire des trajectoires fiables (comme illustré avec la balle de ping-pong). Néanmoins, sa nature épars ne permet ni de segmenter les objets en mouvement, ni de modéliser des déformations continues.

La résilience de l'approximation polynomiale (Farnebäck) : Cette approche dense s'est montrée particulièrement robuste face aux textures non rigides et aux fluides (jet d'eau). En modélisant les voisinages par des polynômes plutôt que par de simples différences finies, elle absorbe beaucoup mieux les variations locales d'illumination.

Le double tranchant de la méthode Horn-Schunck Multi-échelle : Notre implémentation variationnelle globale a brillé sur les solides rigides, offrant une segmentation parfaite et une excellente compréhension de la cinématique complexe (capture de la rotation de la balle). De plus, l'architecture en pyramide d'images a parfaitement résolu la limitation liée aux grands déplacements. Cependant, nos tests sur les fluides ont révélé la faiblesse de ce modèle : sa forte sensibilité aux reflets (faux mouvements) et le *sur-lissage* des discontinuités (le mouvement de la tasse qui "bave" sur les bords statiques).

L'approche variationnelle de Horn-Schunck sur nos tests prouvent que la régularisation de Tikhonov (basée sur la norme L^2) est inadaptée aux frontières nettes des objets. Elle pénalise trop fortement les variations brusques. Pour pallier ce problème et préserver les discontinuités de mouvement, il est nécessaire de modifier la fonctionnelle d'énergie en remplaçant la pénalité quadratique par une norme L^1 (Variation Totale), une extension que nous développons dans la section suivante.

6 Extension : Préservation des discontinuités par la Variation Totale (Norme L^1)

6.1 Limites de la régularisation de Tikhonov (L^2)

La formulation originale de Horn-Schunck repose sur une régularisation de type Tikhonov, utilisant la norme L^2 au carré pour pénaliser les variations spatiales du flot optique :

$$E_{HS}(u, v) = \iint_{\Omega} [(I_x u + I_y v + I_t)^2 + \alpha(|\nabla u|^2 + |\nabla v|^2)] dx dy \quad (14)$$

Bien que cette énergie soit strictement convexe et facile à minimiser (conduisant à un système d'équations linéaires résoluble par Jacobi ou Gauss-Seidel), elle présente un défaut majeur en vision par ordinateur. La pénalité quadratique sanctionne excessivement les forts gradients. Par conséquent, aux frontières des objets en mouvement (où le flot passe brutalement d'une valeur à une autre), l'algorithme lisse la transition pour minimiser l'énergie. Cela entraîne un floutage des contours cinématiques et un mouvement qui "déborde" sur les zones statiques, comme observé lors de nos tests sur le tourbillon de café.

Pour surmonter ce problème, une extension naturelle consiste à remplacer la pénalisation quadratique par la norme L^1 , conduisant au modèle de la Variation Totale (TV).

6.2 Défi technique et implémentation Python

Analyse mathématique de la non-différentiabilité

D'un point de vue mathématique, le passage de la norme L^2 à la norme L^1 transforme notre problème en un défi d'optimisation non-lisse. En effet, la fonctionnelle $E_{TV}(u, v)$ contient des termes en valeur absolue dont la dérivée n'est pas définie en zéro.

Par conséquent, les équations d'Euler-Lagrange classiques que nous avons utilisées pour Horn-Schunck ne peuvent plus être appliquées directement. Pour caractériser le minimum de cette nouvelle énergie, il faut abandonner la notion de gradient classique au profit du **sous-différentiel** (issu de l'analyse convexe). Le sous-différentiel de la fonction $f(x) = |x|$ en zéro n'est plus une valeur unique, mais l'intervalle $[-1, 1]$.

Justification de l'utilisation d'OpenCV

La résolution numérique d'un tel problème nécessite des algorithmes d'optimisation avancés. Les méthodes naïves par différences finies divergent ou oscillent autour des discontinuités. La littérature moderne s'appuie sur des méthodes dites *proximales* ou des algorithmes *primal-dual* (comme l'algorithme de Chambolle-Pock) pour traiter ce sous-différentiel.

Le développement, l'optimisation et la parallélisation d'un tel solveur primal-dual en Python pur (NumPy) est assez laborieux. C'est pourquoi, pour valider notre hypothèse théorique sur des vidéos réelles, nous nous appuyons sur l'implémentation *Dual TV-L¹* d'OpenCV. Cette approche nous permet de vérifier la préservation des discontinuités cinématiques tout en bénéficiant de l'optimisation matérielle du langage C++.

Solution retenue : Dual TV-L¹ (OpenCV)

Afin de valider empiriquement le modèle tout en conservant des temps de calcul acceptables, nous avons opté pour l'utilisation d'une bibliothèque compilée et optimisée en C++. Nous nous appuyons sur l'implémentation de l'algorithme *Dual TV-L¹* fournie par le module *optflow* d'OpenCV.

Le script Python ci-dessous constitue la version finale de notre pipeline d'évaluation vidéo. Il intègre un sélecteur permettant de basculer en temps réel entre l'approche multi-échelle L^2 (notre code), l'approche locale (Lucas-Kanade) et l'approche globale TV-L¹.

```
1 import cv2
2 import numpy as np
3
4 def process_video(video_path, algorithm='tv11', resize_width=320):
5     """
6     Lit une video, estime le flot optique image par image et l'affiche.
7     Algorithmes : 'multiscale', 'farneback', 'lucas_kanade', 'tv11'.
8     """
9     cap = cv2.VideoCapture(video_path)
10
11     if not cap.isOpened():
12         print("Erreur: Impossible de lire la video.")
13         return
14
15     ret, frame1 = cap.read()
16     if not ret: return
17
18     # Redimensionnement pour performances
19     h, w = frame1.shape[:2]
20     new_dim = (resize_width, int(h * (resize_width / float(w))))
21     prvs = cv2.cvtColor(cv2.resize(frame1, new_dim), cv2.COLOR_BGR2GRAY)
22
23     # Initialisations spécifiques
24     if algorithm == 'lucas_kanade':
25         feature_params = dict(maxCorners=100, qualityLevel=0.3, minDistance=7,
26                                blockSize=7)
27         lk_params = dict(winSize=(15, 15), maxLevel=2, criteria=(cv2.
28                                TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03))
29         p0 = cv2.goodFeaturesToTrack(prvs, mask=None, **feature_params)
30         mask_lk = np.zeros_like(cv2.resize(frame1, new_dim))
31         color = np.random.randint(0, 255, (100, 3))
32
33     elif algorithm == 'tv11':
34         # Initialisation de l'algorithme TV-L1 (necessite opencv-contrib-python)
35         tv11_algo = cv2.optflow.createOptFlow_DualTVL1()
36
37     while True:
38         ret, frame2 = cap.read()
```

```

37     if not ret: break
38
39     frame2 = cv2.resize(frame2, new_dim)
40     next_frame = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)
41     vis_frame = frame2.copy()
42
43     # Calcul du flot selon l'algorithme choisi
44     if algorithm == 'tv11':
45         flow = tv11_algo.calc(prvs, next_frame, None)
46         display_img = visualize_flow(flow)
47
48     elif algorithm == 'multiscale':
49         u, v = multiscale_horn_schunck(prvs, next_frame, num_levels=3, alpha
=1.0)
50         display_img = visualize_flow(np.stack((u, v), axis=-1))
51
52     elif algorithm == 'lucas_kanade':
53         if p0 is not None:
54             p1, status, err = cv2.calcOpticalFlowPyrLK(prvs, next_frame, p0,
None, **lk_params)
55             good_new, good_old = p1[status == 1], p0[status == 1]
56             for i, (new, old) in enumerate(zip(good_new, good_old)):
57                 a, b = new.ravel().astype(int)
58                 c, d = old.ravel().astype(int)
59                 mask_lk = cv2.line(mask_lk, (a, b), (c, d), color[i].tolist
(), 2)
60                 vis_frame = cv2.circle(vis_frame, (a, b), 5, color[i].tolist
(), -1)
61                 display_img = cv2.add(vis_frame, mask_lk)
62                 p0 = good_new.reshape(-1, 1, 2)
63             else:
64                 display_img = vis_frame
65
66     # Affichage temps reel
67     combined = np.hstack((frame2, display_img))
68     cv2.imshow(f'Flot Optique - {algorithm}', combined)
69     prvs = next_frame
70
71     if cv2.waitKey(1) & 0xFF == ord('q'):
72         break
73
74     cap.release()
75     cv2.destroyAllWindows()

```

Listing 3 – Pipeline d'évaluation vidéo intégrant l'algorithme TV- L^1

6.3 Évaluation empirique du modèle TV- L^1 et comparaison avec le modèle L^2

Afin de valider l'apport de la Variation Totale (norme L^1), nous avons soumis l'algorithme *Dual TV- L^1* d'OpenCV aux trois mêmes séquences vidéo (balle, café, jet d'eau) qui avaient mis en difficulté notre implémentation de Horn-Schunck (norme L^2). Les résultats sont présentés dans les figures 12, 13 et 14.

Analyse comparative (L^1 vs L^2)

L'observation de ces nouveaux champs de vecteurs confirme toutes les prédictions mathématiques liées au changement de norme de régularisation :

- **Disparition de l'effet de "Halo" (Préservation des discontinuités)** : Le cas du tourbillon de café (Figure 13) est le plus probant. Avec la norme L^2 , le mouvement du



FIGURE 12 – Balle ($TV-L^1$)

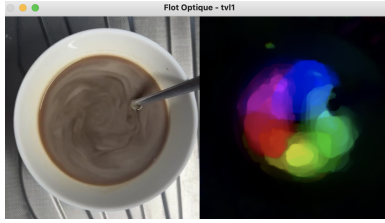


FIGURE 13 – Café ($TV-L^1$)

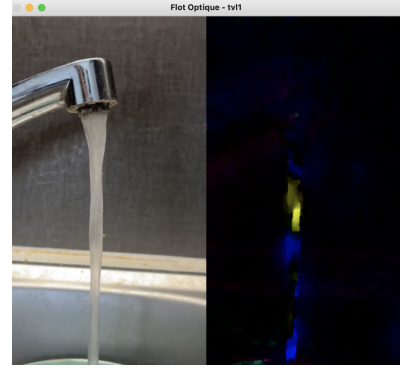


FIGURE 14 – Eau ($TV-L^1$)

liquide "bavait" sur la porcelaine de la tasse pour minimiser l'énergie de gradient. Avec la norme L^1 , le bord blanc de la tasse est parfaitement noir (statique). La Variation Totale a autorisé un saut brutal du flot optique à la frontière géométrique, segmentant parfaitement le fluide du récipient.

- **Robustesse aux reflets et au bruit (Outliers)** : Le cas du robinet (Figure 14) montre également une grande amélioration. Là où Horn-Schunck (L^2) générerait d'immenses traînées parasites sur le métal chromé à cause des reflets, $TV-L^1$ reste imperturbable. Le métal et le fond sont noirs, seul le jet d'eau est capturé. Cela s'explique car l'algorithme *Dual TV-L¹* utilise également la norme L^1 pour le terme d'attache aux données (data term), ce qui le rend extrêmement robuste aux variations locales d'illumination, agissant comme un estimateur robuste (médiane) plutôt qu'une moyenne sensible aux valeurs aberrantes.
- **Le phénomène "Constant par morceaux"** : Sur la balle de ping-pong (Figure 12) et le café, on remarque que les couleurs du flot forment des "taches" ou des aplats très nets, contrairement au dégradé parfaitement lisse obtenu avec L^2 . C'est la signature visuelle typique de la Variation Totale : elle a tendance à créer des régions où le flot est constant, séparées par des frontières nettes. La géométrie de l'objet est parfaitement extraite par rapport au fond en parquet.

Les expérimentations valident l'approche $TV-L^1$ comme étant largement supérieure à la formulation originale de Horn-Schunck pour des scènes réelles. En abandonnant la régularisation de Tikhonov au profit de la Variation Totale, on résout le problème du sur-lissage, on préserve l'intégrité géométrique des objets en mouvement, et on gagne une robustesse indispensable face aux reflets et aux violations de l'hypothèse de constance d'intensité.

Conclusion

Ce projet nous a permis d’explorer en profondeur la problématique de l’estimation du flot optique, un pilier fondamental de la vision par ordinateur. À travers l’étude et l’implémentation de différents modèles, nous avons pu confronter la rigueur de la modélisation mathématique aux contraintes imprévisibles du monde réel.

Dans un premier temps, l’étude de la méthode de Horn-Schunck nous a familiarisés avec l’approche variationnelle globale. Nous avons mis en évidence le rôle crucial du paramètre de régularisation α : un faible lissage préserve les détails mais laisse passer le bruit, tandis qu’une régularisation excessive (norme L^2) entraîne un phénomène de sur-lissage pour la précision des contours cinématiques. La comparaison avec la méthode locale de Lucas-Kanade a souligné l’opposition classique entre flot dense et flot épars, le second offrant une robustesse accrue sur les zones texturées au prix d’une perte d’information sur la structure globale des objets.

L’implémentation de la stratégie multi-échelle a constitué une étape charnière. Nos tests sur les boîtes de pizza et la balle de ping-pong ont démontré que la linéarisation de l’équation de contrainte est insuffisante face aux dynamiques rapides. La pyramide d’images s’est révélée indispensable pour capturer les grands déplacements tout en stabilisant le calcul du flot résiduel par recalage d’image.

Enfin, l’extension vers la Variation Totale ($TV-L^1$) a apporté la solution aux limites observées lors de nos tests sur les fluides et les bords de tasse. En remplaçant la pénalité quadratique par une norme L^1 , nous avons pu valider empiriquement la capacité de cette norme à autoriser des sauts brusques dans le champ de vecteurs, préservant ainsi les discontinuités naturelles du mouvement tout en ignorant les reflets parasites.

En conclusion, ce travail démontre que l’estimation du flot optique est un arbitrage constant entre fidélité aux données et régularité spatiale. Si les méthodes traditionnelles comme Horn-Schunck posent des bases théoriques élégantes, c’est l’évolution vers des modèles non-lisses et multi-échelles qui permet aujourd’hui d’atteindre la précision nécessaire aux applications modernes, de la robotique autonome à l’analyse médicale.